✓ "Vendors already have dashboards — why use yours?"

Objection handling, the tiered-access case & real-world scenarios

THE REFRAME (SAY THIS FIRST)

A vendor's built-in dashboard answers **"how is MY product doing?"** — it is single-vendor, full-access, point-in-time, and locked inside one console behind one login and one permission model. This tool answers **"how is our WHOLE estate doing, across every vendor and site, for the people who must NOT have full console access?"** They do different jobs. You don't replace the vendor consoles — you add the oversight layer none of them can provide.

> **Analogy:** every bank has an app showing your balance *at that bank*. A net-worth aggregator shows **all your banks in one place** and lets your accountant see a **read-only** summary without your banking passwords. You don't stop using the banks — you add the aggregator. This dashboard is that aggregator for endpoint security.

WHY THE BUILT-IN VENDOR DASHBOARD IS not ENOUGH

VENDOR LIMITATION	WHY IT MATTERS	HOW THIS TOOL ANSWERS IT
Single-vendor silo	Defender + CrowdStrike together show no combined fleet or single total	Normalized view across all tools; real org-wide totals
No cross-site rollup	Separate consoles/tenants per office or client → no "whole estate" number	Sites + All-sites aggregate in one screen
Access = console access	Seeing the dashboard needs a console account that usually also grants response, policy & raw-data powers	A Viewer role showing posture only — no console powers
Coarse / over-privileged RBAC	"Read" roles bundled with sensitive abilities; weak per-client scoping	Purpose-built Admin / Analyst / Viewer + per-site scope
Per-seat cost & credential sprawl	10 staff × 5 consoles = 50 accounts to license, secure, rotate, offboard	One dashboard login per person; far fewer vendor seats
Inconsistent "compliant"	Every vendor defines healthy/compliant differently → no uniform KPI	One normalized definition + your own custom rules
Fixed buckets, no custom logic	Can't uniformly define "seen <24h AND version ≥ X"	Rule engine: any field, any threshold, across all tools
No cross-tool history	Each console keeps its own retention; no unified trend line	Snapshots → normalized trends over time
Reporting friction	Board/audit report = N exports + manual merge, stale on arrival	One screenshot / one CSV across everything
Auditor/client access unsafe	An EDR-console seat for an auditor is risky and expensive	Scoped read-only login (or export) — no console seat
Bigger attack/insider surface	More people in the EDR console = more credentials to phish/abuse	Read-only aggregator + "key never stored" option shrinks who needs console creds

Even with a **single** EDR you still gain cross-site rollup, custom rules, role-restricted access for L1/auditors/clients, normalized history, and unified reporting. With multiple vendors the gap is decisive.

THE "IT'S JUST ANOTHER LOGIN" OBJECTION

This objection argues *for* the tool once you do the math:

- **Setup is one-time, by an admin** — connect each source once. Not a per-user, per-session burden.
- **Users sign into ONE dashboard** (with MFA) instead of juggling many vendor logins every shift, forever.
- **Net credentials go DOWN** — one aggregator login can replace multiple vendor-console seats per person.
- **"Enter key every login" is optional** and only for the most sensitive sources; most refresh automatically.

Say it: "The one-time setup replaces a *forever* hassle — every analyst logging into every console, every shift. We trade one setup for fewer logins, fewer seats, fewer credentials to secure."

FLAGSHIP SCENARIO — TIERED ACCESS (L1 MONITORING WITHOUT THE KEYS)

THE KEY USE-CASE **Let L1 monitor everything — without console access to anything.**

Situation. An MSP/SOC (or central security team) monitors many clients/sites, each with its own EDR/XDR tenant. **L1 analysts** provide first-line 24/7 monitoring. Giving every L1 a login to **every client's vendor console** is unacceptable:

- Console access usually includes **response & configuration powers** (isolate a host, kill processes, change policy, pull sensitive data) — far more than "look at health".
- It **violates least privilege** and often breaches **client contracts / data-segregation** rules.
- It **multiplies credentials** (every L1 × every client × every vendor) — a provisioning, rotation & offboarding nightmare and a large insider/breach surface.
- Vendor RBAC often **can't scope cleanly** to "this client, read-only, compliance only".

With this dashboard. L1 gets a **single Viewer login** and sees **scoped, read-only** compliance and health for only the sites/clients assigned — **no response actions, no policy access, no raw console**. They watch dashboards and **custom rules** ("offline > 2 days", "agent outdated", "non-compliant spike").

When something is off. L1 **raises a ticket / informs a senior (L2/L3)** who *does* hold console access to investigate and act. **L1 never needs console credentials** to do their monitoring job.

Compliance reporting. Clients, auditors or managers who **should not** have the security console get a **Viewer login to just their site** — or an exported report — proving posture **without exposing the full console**.

Why it's better

- Least privilege, enforced by the tool
- Fewer seats & credentials
- Clean separation of duties (audit-logged)
- Faster, safer triage & escalation
- Client-safe transparency
- Full record of who viewed/changed what

One-liner: "We can't give L1 (or clients) access to every vendor console — that hands out response powers and breaks least privilege. So we built a read-only, scoped monitoring + compliance layer: L1 watches, raises tickets, and escalates; seniors with console access act. Everyone sees the truth; only the right people can touch the controls."

MORE SCENARIOS

1. **Auditor self-serve.** A Viewer login or CSV of historical compliance — no costly, risky EDR seat.
2. **Board report in 60s.** Open All-sites, screenshot the compliance trend + OS breakdown.
3. **Multi-vendor migration.** Run CrowdStrike + SentinelOne at once; prove parity before cutover.
4. **The silent endpoint.** "Last seen > 7 days" surfaces lost/compromised hosts across all tools.
5. **Patch campaign.** Agent-version drift list → a precise upgrade target, not a guess.
6. **Client portal.** Each client gets a Viewer login scoped to their site only.
7. **After-hours coverage.** Night-shift L1 watches many tenants from one screen; escalates by ticket.
8. **Separation of duties.** Who monitors ≠ who changes — enforced by role, evidenced by audit log.
9. **Fast onboarding.** New analyst gets a Viewer login day one — no provisioning 5 consoles.
10. **Clean offboarding.** Disable one account, not hunt accounts across N consoles.
11. **Cost control.** No extra vendor seats for read-only managers/auditors/clients.
12. **Feed-health awareness.** System Health flags a failing/stale source instead of false "all clear".
13. **SLA reporting.** Uniform "% compliant per site/client" as SLA evidence.
14. **Incident retro.** Snapshots show posture before/after an incident.
15. **M&A / new site.** Connect the new org's API once → instantly in All-sites totals.

WHEN THE VENDOR CONSOLE IS THE RIGHT TOOL (HONESTY BUILDS TRUST)

State this proactively — it pre-empts "but the console does more" and makes your case credible. The vendor console is the right place for:

- **Deep investigation & threat hunting** in raw telemetry
- **Response actions** — isolate a host, kill a process, remediate, roll back
- **Policy configuration** and detailed alert triage

This dashboard **deliberately does none of those**. It is a **coverage, compliance & oversight** layer that *complements* the consoles. The right answer is almost always **"both"**: consoles for action, this tool for the unified, role-restricted, historical picture.

DECISION MATRIX

NEED	VENDOR CONSOLE	THIS DASHBOARD
Investigate an alert / threat hunt	✓	—
Isolate / remediate a host	✓	—
Configure detection policy	✓	—
Single number across **all** vendors	—	✓
One view across **all sites / clients**	—	✓
Read-only access for L1 / auditors / clients	⚠ over-privileged	✓ **scoped**
Custom compliance rules & thresholds	—	✓
Cross-tool history & trends	—	✓
One-click board / audit report	⚠ manual	✓
Fewer credentials & seats to manage	—	✓

OBJECTION-HANDLING CHEAT SHEET — "IF THEY SAY… YOU SAY…"

"Vendors already have dashboards."
→ For operating *that one product*, yes. They can't give a cross-vendor, multi-site, role-restricted, historical compliance view — a different job, and the one the team and auditors actually need.

"It's just another login."
→ One login replaces many. Setup is one-time per source; users stop juggling consoles every shift, and we manage far fewer seats and credentials overall.

"Why not just grant read-only console access?"
→ Vendor "read-only" is coarse and often bundled with response powers, costs a seat each, multiplies credentials, can't scope cleanly per client — and is still single-vendor. A scoped Viewer here is safer, cheaper, unified.

"Is the aggregator itself a new risk?"
→ It's built to the standards it monitors: AES-256-GCM-encrypted keys (with a never-stored option), Argon2id + MFA, IP flagging, RBAC, and a full audit log — and it *reduces* how many people need console credentials.

"We only use one EDR."
→ You still get cross-site rollup, custom rules, role-restricted access for L1/auditors/clients, normalized history, and one-click reporting — and you're future-proofed the day you add or switch a vendor.

TL;DR — Vendor dashboards run their product; this tool runs your oversight. It unifies every vendor and site into one normalized, historical, role-restricted, audit-ready view — so L1 can monitor and escalate **without** the keys to every console, auditors and clients can see posture **without** a security-console seat, and leadership gets a single compliance number with proof over time. Keep the consoles for action; add this for visibility, control, and trust.

Companion docs: docs/VALUE-JUSTIFICATION.md · docs/value-summary.pdf · docs/CONNECTING-SOURCES.md